

Designing an App that Recognizes the Facial Markers of Autism Spectrum Disorder

Word Count: 4810

Abstract

In the United States, 1 in 36 children have Autism Spectrum Disorder. Despite the high number of children with this disorder, Autism Spectrum Disorder is a disorder that goes heavily undiagnosed. A fourth of autistic children continue to remain undiagnosed today. When a child receives a late diagnosis, the opportunity to take advantage of important resources is greatly diminished. Many studies strive to find more efficient methods of ASD diagnosis, yet the majority of these studies are inaccessible to the public. From MRI machines to in-depth kinematic analyses, these studies are either too expensive or too complicated for most people to access. This study was carried out as an attempt to overcome the inaccessibility of autism diagnosis. A database of 2,540 images, Vgg16, Anaconda Navigator, and the Python coding language were all obtained to begin the study. These resources were used to create an app that uses facial recognition technology to recognize autism in children up to 11 years old. Python was used to program and train Vgg16, a machine-learning model that uses data to learn. The model was programmed on the Kaggle Database, which was also where the set of images was obtained. After the model was trained, a series of 3 test runs took place to note the accuracy of the program. Finally, Vgg16 was combined with Anaconda Navigator to place the model into the format of a mobile app. During the training process, the training accuracy showed a steady increase. The highest training accuracy the model reached was 96%. The findings of this study suggest that facial recognition technology can be used to create an accessible app that accurately detects Autism Spectrum Disorder. Technology like this can be used to improve the ASD diagnosis rate throughout the United States.

Literature Review

Introduction

Autism Spectrum Disorder is a disorder that affects social skills and learning. This genetic disorder is commonly associated with children¹. A study published by the CDC shows that in the United States, 1 in 36 children have Autism Spectrum Disorder¹. Despite this large statistic, a fourth of autistic children remain undiagnosed today². Several authors within the *Journal of Child Psychology and Psychiatry* recently released a work explaining that when autistic children receive a late diagnosis they miss out on many resources that greatly improve their ability to regulate emotions³. The most likely cause for the extreme lack of diagnosis is the amount of time pediatricians spend during the first appointments of a child's life. A study done by Neal Halfon, Gregory D. Stevens, Kandyce Larson, and Lynn M. Olson, authors published in the *AAP Pediatrics* journal, shows that nearly half of reported pediatricians spend less than twenty minutes in these crucial appointments⁴. To properly recognize the signs of autism in a child, a professional would need to spend hours in one appointment alone. In these appointments, health professionals have to pay attention to not only the behavioral patterns of a child, but the physical aspects as well. Even though these particular aspects often go unnoticed, they could be used to increase the rate of diagnosis among autistic children if properly taken into account.

Technology and Autism

One review published in the *Journal of Bioengineering* capitalizes on this fact. The authors of the review, who all studied at the Peking Union Medical College Hospital in Beijing, researched the use of facial recognition-based applications concerning disease diagnosis. While it is important to acknowledge that autism is not a disease, this review heavily applies to autistic children⁵. It is known that autistic children make different or otherwise inappropriate facial

expressions than typically developing children. Not only that, autistic children face difficulty in understanding facial expressions that their peers may make¹. Recent studies published by the National Library of Medicine also explain that autistic children have contrasting facial features compared to typically developing children⁶. This means that the technology in the study published by the authors at Peking Union Medical College Hospital could be used to recognize facial features that present differently in an autistic child than a typically developing child. The study presented by the researchers at Peking Union Medical College Hospital is not the only breakthrough within the realm of using technology to detect Autism Spectrum Disorder in children. Studies published by Roberta Simeoli, Nicola Milano, Angelo Rega, and Davide Marocco in the *Frontiers in Psychology* journal as well as K. K. Mujeeb Rahman—who has many publications within the realm of technology and autism research—and M. Monica Subashini—who frequently studies imaging technology related to health and engineering—from the *Journal of Brain Sciences*⁶⁻⁷ used different technological methods to recognize autistic children. The authors of the first document used a tablet to perform a kinematic analysis method combined with a supervised machine learning method to determine if a child showcased motor patterns common in Autism Spectrum Disorder⁷. The test was first performed using the tablet, then the results were gathered to create datasets to feed into the pre-trained model. In the second publication, the researchers used a set of deep neural networks to detect autism in static images. Instead of generating new images using advanced technology, the researchers used images that already existed to test new technology. Despite some differences within the methods used to accomplish the goals of the projects, these researchers used machine learning models and data processing methods to analyze the facial distinctions commonly seen in autistic children. The use

of machine learning models in both of these reviews shows a recurring theme in the current realm of study regarding alternative methods of Autism Spectrum Disorder diagnosis.

User Accessibility and Disability

Researchers in this field have covered a variety of ways to use technology to detect autistic features. However, most of these techniques are not available to the average person. A prime example of this is a study that makes use of an MRI machine⁸. The study, conducted by members of both the Psychology department and the School of Computing and Informatics at Florida International University, utilizes machine learning imaging techniques in order to diagnose Autism Spectrum Disorder as well as Attention Deficit Hyperactivity Disorder⁸. The MRI is used to produce images to feed into pre-trained machine learning models. The models then use these images to develop datasets on which the models perform tests. Many people today cannot access an MRI machine during a medical emergency, let alone access one for a diagnostic process. MRI diagnosis isn't the only example of this issue. A recent survey hosted by CMS Health Equity Conference states that over 62% of respondent health centers have autism assessment waitlists that are over 6 months long⁹. During a period of that length, young children quickly go through many developmental phases. As an autistic child grows older and hits these developmental milestones, any occupational therapy or other resources begin to lose their effectiveness. As a result, long assessment waitlists inadvertently take many opportunities away from autistic children. Diagnosis accessibility as a whole is a recurring problem within the autistic community, which means that creating a method that is accessible to a large population should be a priority for the researchers. Autism and accessibility are two topics that often mesh together, as autistic children are likely to need accommodations in their daily lives. Two informational programs at the University of Washington released a study regarding the topic of

disability and the use of mobile devices as accommodations. Researchers performing this study collected data using a variety of methods, including surveying participants and requesting written entries relating to the participants' weekly accommodation use. The work concludes that the majority of disabled people carry a smartphone or other mobile device with them in their day-to-day life¹⁰. With this information, it can be inferred that disabled people receive substantial benefits from these devices. However, the devices have more than just a single target audience. These appliances can also accommodate or otherwise positively impact medical professionals as well. Health professionals have included mobile devices in their work in recent years. This can be seen in an analysis published by C. Lee Ventola, who has released several scholarly articles in the National Library of Medicine. Ventola's analysis not only explains the history of device use in professional health settings, it also expertly provides details on the advantages of apps in the field¹¹. These apps have led to many technological advancements within medical software. Examples like the ones previously listed show that many people rely on mobile phones as technological accommodations, meaning that the ability to access and use the accommodating software on the phone is crucial. The largest factor that can affect the accessibility of a mobile app is the design of the interface. A variety of studies throughout the world have been released regarding accessibility and application interface, including one written by a group of researchers from Brazil¹². The work acknowledges the importance of mobile devices in daily life before going on to discuss methods for removing the accessibility barrier in mobile interfaces. Researchers in the study gathered preliminary information before creating a set of guidelines used to test interface design patterns that may allow for more user accessibility in disabled populations. Accessibility isn't only examined in a broad sense as some reviews have mentioned before. Scientists have also evaluated the public use of apps that take advantage of the sensors

commonly located in smartphones to maintain optimal health conditions¹³. Results of the evaluation state that there is room for an even larger amount of opportunities within this field. Opportunities provided by the creation of a healthcare app should be largely taken advantage of because they allow a wider range of people, disabled and able-bodied, to have increased access to modern healthcare and diagnosis technology. Making use of an app that could suggest further screening, as some of the apps mentioned throughout this review do, will potentially lower the average age of diagnosis not only for autistic children but children with other disabilities as well.

Gap

This study was designed to address a gap in data relating to autism diagnosis with machine learning. Despite many lengthy studies that review methods of diagnosis using machine learning, there was a significant lack of data involving the use of mobile apps in the diagnosis process. Some methods of applying this technology came close to the subject, including one study that temporarily hosted a web browser to use their model. However, this web browser has since been taken down and is no longer available to the public as it was developed for testing purposes only. The researchers also did not specify if the web browser would be capable of running on a mobile device. It is important to address the gap in this field of research because the implementation of mobile apps in the realm of autism diagnosis would largely increase the accessibility of autism diagnosis. It is also important to find more data on this part of the research because there is a large chance of varying accuracy rates between alternate methods of hosting these models compared to a mobile app.

Methodology

Procedure

The first step was to obtain a Python programming language development environment program for Windows. Anaconda Navigator was used due to its extreme flexibility in a native Windows system, and it seamlessly incorporates the Jupyter Notebook. Jupyter Notebook is the coding interface used on Kaggle¹⁴. Kaggle is an online scientific model analysis, development, and prototyping website specializing in neural network AI modeling. Pre-trained models of each type were imported into several training applications. Each model is pre-trained on the ImageNet set of training images for general classification purposes. Each model is then customized with further training through a publicly available autism image data set consisting of 2,540 images of individuals with and without autism. For model validation, a separate data set of 100 images is used to verify model performance after training. A test data set of 300 images is used to test the model predictions. All images in the dataset are labeled autistic or non-autistic for training and verification purposes. Model training was performed in a Python coding environment on the Kaggle development platform. Kaggle is well suited to this due to the training routines being executed on Kaggle hardware, which is much more substantial than the personal computers used in the rest of this development. Image processing was done using Tensorflow¹⁵ and Keras¹⁶ API's.

Program creation

The first step in coding an application in the Python programming language is to import all the required libraries into the program. Libraries are specific routines that accomplish various tasks in a Python program. The libraries imported to the training program are shown below:

Importing necessary libraries

import pandas as pd

```
import numpy as np

import cv2

import matplotlib.pyplot as plt

import matplotlib.image as mpimg

%matplotlib inline

import seaborn as sns

import random

import os

import gc


from tensorflow import keras

from tensorflow.keras import layers

from tensorflow.keras import models, Sequential

from tensorflow.keras import optimizers

from tensorflow.keras.optimizers import RMSprop, Adam

from tensorflow.keras import optimizers


from keras.layers.core import Dense, Dropout, Flatten

from keras.layers.convolutional import Conv2D, MaxPooling2D, SeparableConv2D


from tensorflow.keras.applications.vgg16 import VGG16


from keras.preprocessing.image import ImageDataGenerator
```

```
from keras.preprocessing.image import img_to_array, load_img
```

```
from sklearn.metrics import accuracy_score
```

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
from sklearn.metrics import roc_curve, auc
```

```
from sklearn.metrics import precision_recall_curve
```

Dataset preparation

The dataset contains images that are of different sizes. For the model to correctly analyze the image and develop the needed classifications, the images must be scaled to the same size.

This is accomplished using the ImageDataGenerator call in Python. The values used are as follows:

```
# Set the dimensions for images
```

```
nrows = 224
```

```
ncolumns = 224
```

```
channels = 3
```

```
# Read and process the images: Function returns X,y. X - list of resized images, y - list of labels for the images
```

```
def read_and_process_image(list_of_images):
```

```
    X = []
```

```
    y = []
```

```

        for image in list_of_images:

X.append(cv2.resize(cv2.imread(image, cv2.IMREAD_COLOR), (nrows, ncols), interpolation =
        cv2.INTER_CUBIC))

        if 'Non_Autistic' in image:

            y.append(0)

        else:

            y.append(1)

    return X,y

# Get resized images and labels from train data

X_train, y_train = read_and_process_image(train_imgs)


# Convert the lists to array

plt.figure(figsize=(12, 8))

X_train = np.array(X_train)

y_train = np.array(y_train)

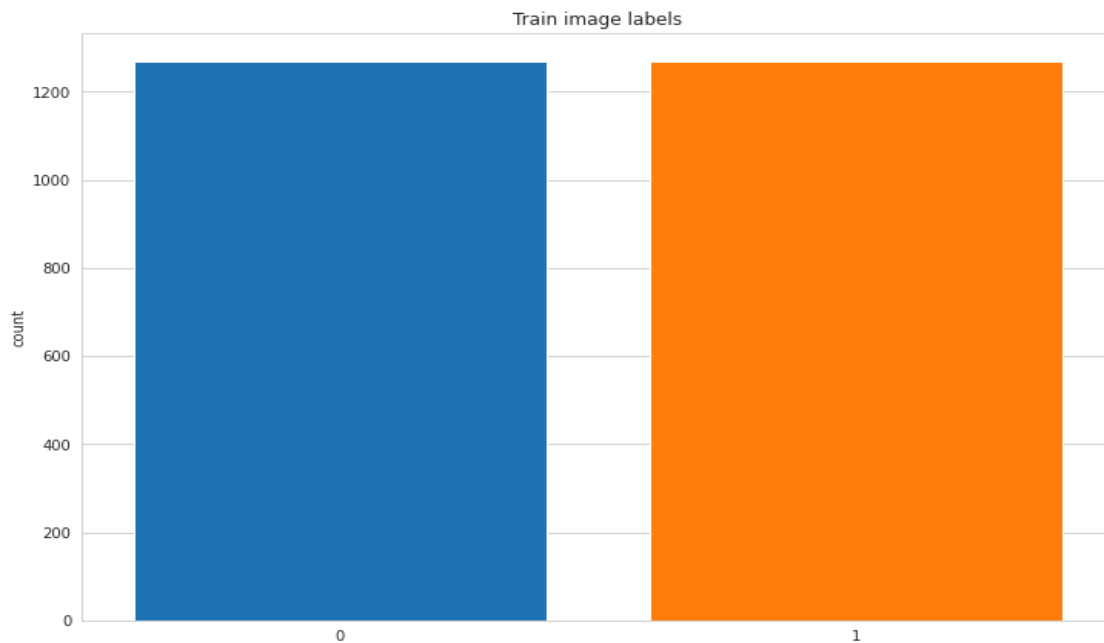
sns.countplot(y_train, saturation=1)

plt.title("Train image labels")

```

A graph can then be plotted showing the distribution of images and their classifications used to train the model, as shown below in Figure 1.

Figure 1. Train Image Labels



After the images have been resized, image batch sizes are defined. A value of '1' is applied to autistic children, and a value of '0' is applied to autistic children. This graph not only shows that there is an even distribution between autistic and non-autistic subjects, but this attribute also tells the model how many images it is processing during each training run and the validation tags it needs to determine model accuracy during and after training. The code is shown below:

```
# Get length of train data and validation data
```

```
ntrain = len(X_train)
```

```
nval = len(X_val)
```

```
batch_size = 16
```

The next step is loading image directories and defining the directory names used throughout the program. There are 3 primary directories in the model training application.

Getting 'Autistic' and 'Non-Autistic' train images from respective file names of train data

```
train_non_autistic = []  
  
train_autistic = []  
  
for i in os.listdir(train_dir):  
  
    if 'Non_Autistic' in ("../input/autism-image-data/AutismDataset/train/{}".format(i)):  
train_non_autistic.append("../input/autism-image-data/AutismDataset/train/{}".format(i))  
  
    else:  
  
train_autistic.append("../input/autism-image-data/AutismDataset/train/{}".format(i))
```

Getting test images from test data file path

```
test_imgs = ["../input/autism-image-data/AutismDataset/test/{}".format(i) for i in  
os.listdir(test_dir)]
```

Concatenate 'Autistic' and 'Non-Autistic' images and shuffle them as train_images

```
train_imgs = train_autistic + train_non_autistic  
  
random.shuffle(train_imgs)  
  
val_autistic = "../input/autism-image-data/AutismDataset/valid/Autistic"  
  
val_non_autistic = "../input/autism-image-data/AutismDataset/valid/Non_Autistic"
```

```

val_autistic_imgs = ["../input/autism-image-data/AutismDataset/valid/Autistic/{}".format(i) for i
                    in os.listdir(val_autistic)]

val_non_autistic_imgs =
["../input/autism-image-data/AutismDataset/valid/Non_Autistic/{}".format(i) for i in
os.listdir(val_non_autistic)]

val_imgs = val_autistic_imgs + val_non_autistic_imgs

random.shuffle(val_imgs)

```

The CNN model is the next object written in the application. It is created and defined with the code in the example shown.

```

# Calling pre-trained VGG16 model

base_model = VGG16(include_top=False, weights='imagenet', input_shape=(224,224,3))

for layer in base_model.layers:

    layer.trainable = False

print("Number of layers in the base model: ", len(base_model.layers))

# Create our classifier model, connect pre-trained model vgg to our model

model = keras.models.Sequential()

model.add(base_model)

model.add(layers.Flatten())

model.add(layers.Dense(4096, activation = 'relu'))

model.add(layers.Dropout(0.2))

```

```
model.add(layers.Dense(4096, activation = 'relu'))  
  
model.add(layers.Dropout(0.2))  
  
model.add(layers.Dense(1, activation = 'sigmoid'))  
  
# Create summary of our model  
  
model.summary()
```

A VGG16 CNN model pre-trained with image weights using the ImageNet training database was used. This database is used to provide initial image recognition weights and parameters to the model. The model was then programmed with the necessary layers and connected to the pre-trained model. This will allow the newly constructed model to be trained using our autism data set. After this connection is established, a summary of the parameters of our model is printed, shown below.

Model: "sequential_20"		
Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 7, 7, 512)	14714688
flatten_20 (Flatten)	(None, 25088)	0
dense_68 (Dense)	(None, 4096)	102764544
dropout_21 (Dropout)	(None, 4096)	0
dense_69 (Dense)	(None, 4096)	16781312
dropout_22 (Dropout)	(None, 4096)	0
dense_70 (Dense)	(None, 1)	4097
Total params: 134,264,641		
Trainable params: 119,549,953		
Non-trainable params: 14,714,688		

Receiving the printed summary of these parameters concludes the creation of the model.

After this, training data can be created by the model.

Data Analysis

Training Results

Model training was initiated using the autism data set. A total of 119,549,953 parameters were trained on the model using 16 image steps. The model ran 40 evolutions of the data set, with each evolution taking 158 steps. The loss and accuracy percentages during each evolution

were observed to verify continuous model accuracy improvement. The training results are shown below.

Figure 2. Epochs (1)

```
Epoch 1/40
158/158 [=====] - 28s 170ms/step - loss: 0.6426 - acc:
0.6272 - val_loss: 0.5742 - val_acc: 0.6979
Epoch 2/40
158/158 [=====] - 27s 169ms/step - loss: 0.5557 - acc:
0.7139 - val_loss: 0.5734 - val_acc: 0.6875
Epoch 3/40
158/158 [=====] - 27s 168ms/step - loss: 0.5221 - acc:
0.7306 - val_loss: 0.5433 - val_acc: 0.6979
Epoch 4/40
158/158 [=====] - 27s 170ms/step - loss: 0.5129 - acc:
0.7536 - val_loss: 0.5412 - val_acc: 0.7292
Epoch 5/40
158/158 [=====] - 27s 170ms/step - loss: 0.5154 - acc:
0.7409 - val_loss: 0.5345 - val_acc: 0.6771
Epoch 6/40
158/158 [=====] - 27s 170ms/step - loss: 0.5025 - acc:
0.7460 - val_loss: 0.5462 - val_acc: 0.7083
Epoch 7/40
158/158 [=====] - 27s 170ms/step - loss: 0.5023 - acc:
0.7583 - val_loss: 0.5162 - val_acc: 0.6771
Epoch 8/40
158/158 [=====] - 27s 169ms/step - loss: 0.4846 - acc:
0.7599 - val_loss: 0.5033 - val_acc: 0.7083
Epoch 9/40
158/158 [=====] - 27s 170ms/step - loss: 0.4928 - acc:
0.7647 - val_loss: 0.5951 - val_acc: 0.7292
Epoch 10/40
158/158 [=====] - 26s 167ms/step - loss: 0.4956 - acc:
0.7532 - val_loss: 0.5187 - val_acc: 0.6458
Epoch 11/40
158/158 [=====] - 27s 171ms/step - loss: 0.4625 - acc:
0.7777 - val_loss: 0.5156 - val_acc: 0.7188
Epoch 12/40
158/158 [=====] - 27s 169ms/step - loss: 0.4827 - acc:
0.7682 - val_loss: 0.5339 - val_acc: 0.6771
Epoch 13/40
158/158 [=====] - 27s 169ms/step - loss: 0.4621 - acc:
0.7769 - val_loss: 0.5062 - val_acc: 0.7083
Epoch 14/40
```

Figure 3. Epochs (2)

```

158/158 [=====] - 27s 169ms/step - loss: 0.4671 - acc:
0.7742 - val_loss: 0.5326 - val_acc: 0.6875
Epoch 15/40
158/158 [=====] - 27s 172ms/step - loss: 0.4551 - acc:
0.7857 - val_loss: 0.5527 - val_acc: 0.6979
Epoch 16/40
158/158 [=====] - 27s 169ms/step - loss: 0.4467 - acc:
0.7805 - val_loss: 0.5233 - val_acc: 0.7083
Epoch 17/40
158/158 [=====] - 27s 168ms/step - loss: 0.4612 - acc:
0.7817 - val_loss: 0.5019 - val_acc: 0.7083
Epoch 18/40
158/158 [=====] - 27s 171ms/step - loss: 0.4664 - acc:
0.7900 - val_loss: 0.5248 - val_acc: 0.7292
Epoch 19/40
158/158 [=====] - 27s 169ms/step - loss: 0.4515 - acc:
0.7801 - val_loss: 0.4803 - val_acc: 0.7500
Epoch 20/40
158/158 [=====] - 27s 172ms/step - loss: 0.4401 - acc:
0.7952 - val_loss: 0.4996 - val_acc: 0.7396
Epoch 21/40
158/158 [=====] - 27s 170ms/step - loss: 0.4481 - acc:
0.7845 - val_loss: 0.5393 - val_acc: 0.7500
Epoch 22/40
158/158 [=====] - 27s 171ms/step - loss: 0.4333 - acc:
0.8047 - val_loss: 0.4790 - val_acc: 0.7500
Epoch 23/40
158/158 [=====] - 26s 167ms/step - loss: 0.4362 - acc:
0.7924 - val_loss: 0.5188 - val_acc: 0.7188
Epoch 24/40
158/158 [=====] - 27s 170ms/step - loss: 0.4433 - acc:
0.7936 - val_loss: 0.5037 - val_acc: 0.7083
Epoch 25/40
158/158 [=====] - 27s 168ms/step - loss: 0.4332 - acc:
0.7960 - val_loss: 0.5154 - val_acc: 0.6875
Epoch 26/40
158/158 [=====] - 27s 170ms/step - loss: 0.4360 - acc:
0.7932 - val_loss: 0.5767 - val_acc: 0.7396
Epoch 27/40

```

Figure 4. Epochs (3)

```

158/158 [=====] - 27s 168ms/step - loss: 0.4413 - acc:
0.7896 - val_loss: 0.5197 - val_acc: 0.7083
Epoch 28/40
158/158 [=====] - 27s 169ms/step - loss: 0.4072 - acc:
0.8114 - val_loss: 0.5298 - val_acc: 0.7083
Epoch 29/40
158/158 [=====] - 27s 168ms/step - loss: 0.4242 - acc:
0.8043 - val_loss: 0.5492 - val_acc: 0.7292
Epoch 30/40
158/158 [=====] - 27s 169ms/step - loss: 0.4142 - acc:
0.8031 - val_loss: 0.5073 - val_acc: 0.7188
Epoch 31/40
158/158 [=====] - 27s 171ms/step - loss: 0.4209 - acc:
0.8086 - val_loss: 0.4920 - val_acc: 0.7188
Epoch 32/40
158/158 [=====] - 27s 169ms/step - loss: 0.4222 - acc:
0.7964 - val_loss: 0.5226 - val_acc: 0.6771
Epoch 33/40
158/158 [=====] - 27s 169ms/step - loss: 0.4438 - acc:
0.7904 - val_loss: 0.5205 - val_acc: 0.7396
Epoch 34/40
158/158 [=====] - 27s 170ms/step - loss: 0.4218 - acc:
0.8023 - val_loss: 0.5000 - val_acc: 0.7396
Epoch 35/40
158/158 [=====] - 27s 171ms/step - loss: 0.4237 - acc:
0.7979 - val_loss: 0.5278 - val_acc: 0.7083
Epoch 36/40
158/158 [=====] - 27s 170ms/step - loss: 0.4095 - acc:
0.8098 - val_loss: 0.5204 - val_acc: 0.7500
Epoch 37/40
158/158 [=====] - 27s 168ms/step - loss: 0.4057 - acc:
0.8078 - val_loss: 0.5396 - val_acc: 0.7500
Epoch 38/40
158/158 [=====] - 27s 169ms/step - loss: 0.4126 - acc:
0.8035 - val_loss: 0.5155 - val_acc: 0.7500
Epoch 39/40
158/158 [=====] - 27s 169ms/step - loss: 0.4012 - acc:
0.8162 - val_loss: 0.5055 - val_acc: 0.7188
Epoch 40/40

```

Figure 5. Epochs (4)

```

158/158 [=====] - 27s 170ms/step - loss: 0.4004 - acc:
0.8138 - val_loss: 0.5217 - val_acc: 0.7604

```

After each epoch, the user is provided with an accuracy and a loss number under training and validation labels. As the model progressed through each epoch, the accuracy numbers went

up. This shows that the model is capable of being accurate if tweaked with the right settings. As the accuracy went up, the loss value went down. The loss value essentially tracks the amount of wrong predictions the model makes. If the value is higher, there are more mistakes. This being said, a lower value represents fewer mistakes.

Using the results from the epochs, the learning curve was graphed.

```
# Plot train and validation accuracy  
  
plt.figure(figsize=(12, 8))  
  
sns.lineplot(data=history_df.loc[:, ["acc", "val_acc"]], palette=['b', 'r'], dashes=False)  
  
sns.set_style("whitegrid")plt.xlabel("Epochs")plt.ylabel("Accuracy")  
  
plt.title("Training and Validation Accuracy")
```

Figure 6. Training and Validation Accuracy



The results from the graphs help facilitate the use of a quantitative¹⁷ data analysis method. These graphs show the information given after each epoch. As the Y value increases, so does the accuracy. As the X value increases, the amount of epochs at that time also increases. The blue line represents the training accuracy. In this training session, the model showcased a steady accuracy increase. The validation accuracy, plotted in red, does not exhibit an increase that is as smooth as the training accuracy. It often rapidly decreases or hits plateaus. One reason for this may be the disparity between the size of the ‘training’ image set and the ‘validation’ image set. The training set contains 2,540 images. In comparison, the validation set is much smaller with only 300 images. With fewer images to validate with, the validation accuracy

changes more frequently compared to the training accuracy. Following the accuracy graph, the loss was also plotted using the following code.

```
# Plot train and validation loss  
  
plt.figure(figsize=(12, 8))  
  
sns.lineplot(data=history_df.loc[:, ["loss", "val_loss"]], palette=['b', 'r'], dashes=False)  
  
sns.set_style("whitegrid")plt.xlabel("Epochs")plt.ylabel("Loss")  
  
plt.title("Training and Validation Loss")
```

Figure 7. Training and Validation Loss



Continuous improvement in accuracy was demonstrated throughout the training run, however loss stabilized at approximately 0.50-0.55. Similar to the accuracy graph, there is an apparent disparity between the validation loss and the training loss. This can again be attributed to the difference between the validation image set and the training image set. In the future, a larger validation set should be applied to close the gap between the validation numbers and the training numbers. After plotting the accuracy and loss graphs, the model was then tested using randomly generated test images in the autism data set.

```
# Read and resize test images

random.shuffle(test_imgs)

X_test, y_test = read_and_process_image(test_imgs)

X = np.array(X_test)

#test_datagen = ImageDataGenerator(rescale = 1./255)

# Predict label for test images

pred = model.predict(X)

threshold = 0.5

predictions = np.where(pred > threshold, 1,0)

#print(predictions)

# Plot test images and their corresponding predictions

test = pd.DataFrame(data = predictions, columns = ["predictions"])

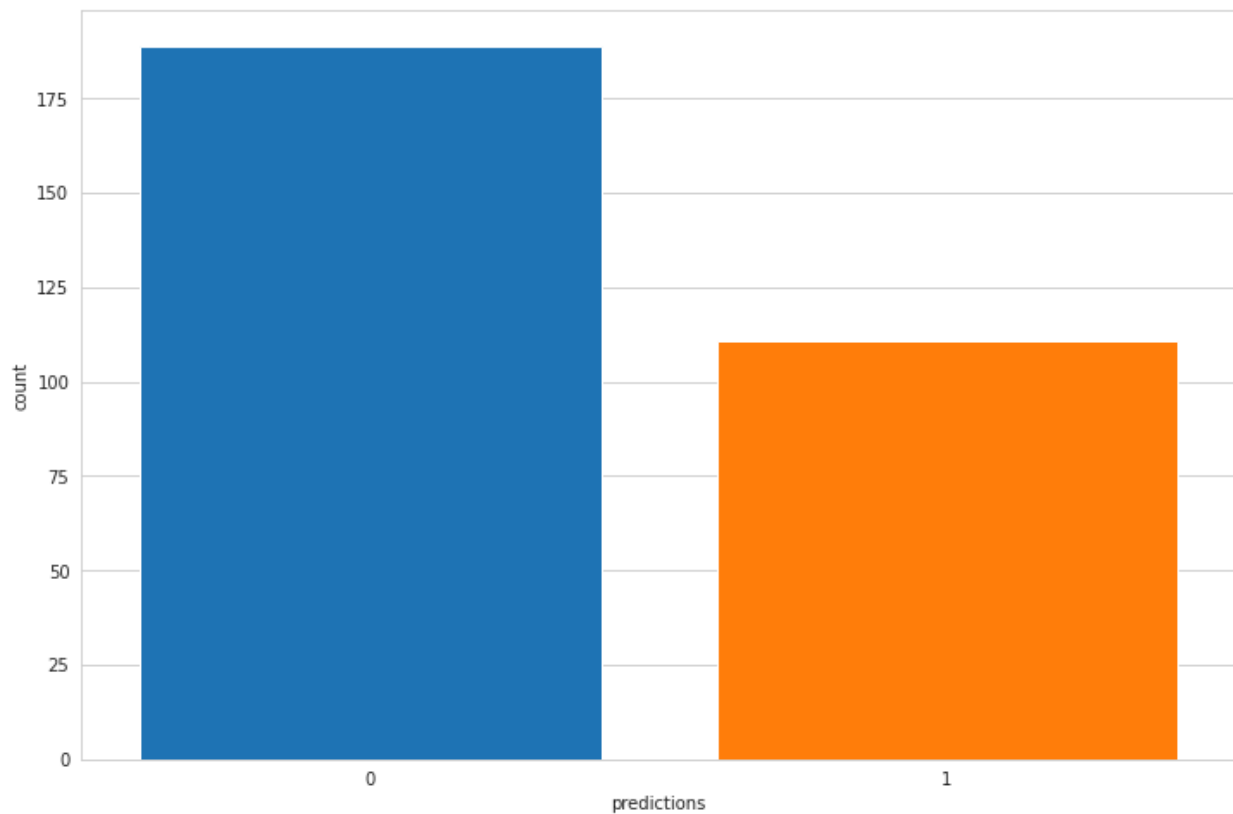
test
```

```
test["filename"] = [os.path.basename(i) for i in test_imgs]
```

```
test["test_labels"] = y_test
```

```
test = test[["filename", "test_labels", "predictions"]]
```

Figure 8. Test Predictions



In this graph, 0 represents a prediction of non-autistic, and 1 represents autistic. The model predicted that there were more non-autistic children than autistic children. This shows that there was a level of inaccuracy, as the images are evenly divided between non-autistic and autistic children. Some example images and model predictions are shown below.

Figure 9. Prediction Example 1

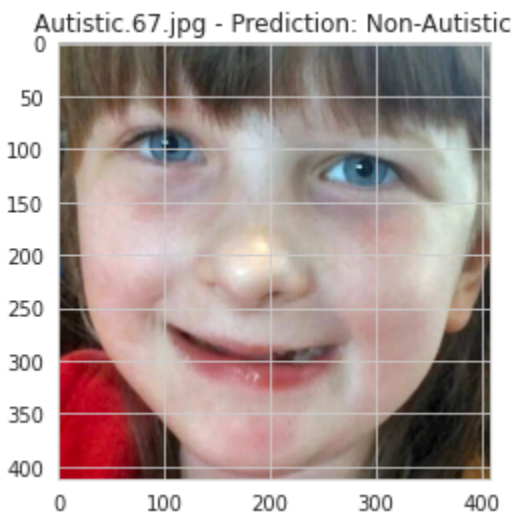
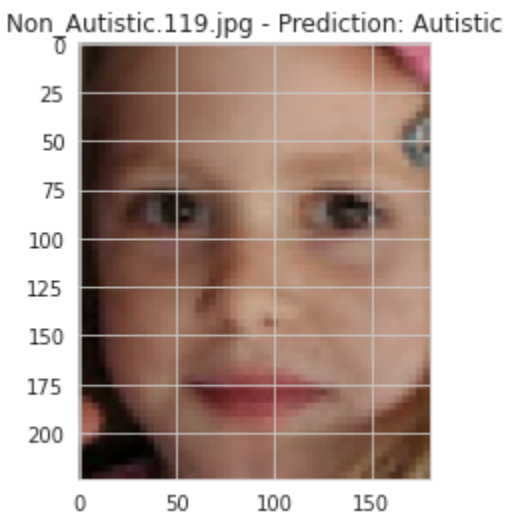


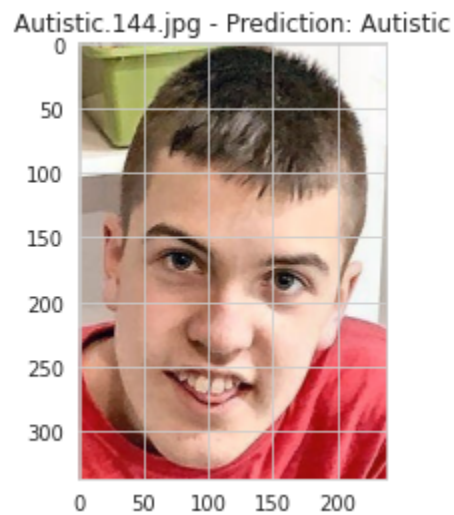
Figure 10. Prediction Example 2



As seen, the prediction provided by the model is placed on top of the image. This prediction can then be confirmed using the title of the file, as the images are labeled 'Autistic' or 'Non_Autistic.' In the case of the two predictions in Figures 9 and 10, the model was incorrect.

This proves that the model is not fully accurate, which means that it still has room for improvement. This, however, does not fully eliminate the connection between machine learning and autism diagnosis.

Figure 11. Prediction Example 3



In this example, the prediction the model exhibited was correct. This is also true for many of the other predictions, which will be showcased in Figures 12-16

Figure 12. Prediction Example 4

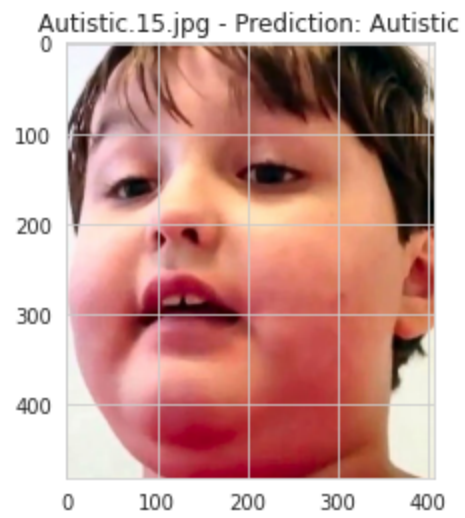


Figure 13. Prediction Example 5

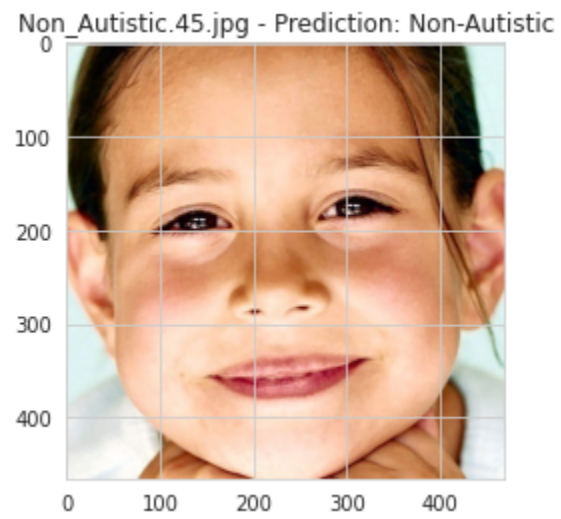


Figure 14. Prediction Example 6

Non_Autistic.123.jpg - Prediction: Non-Autistic

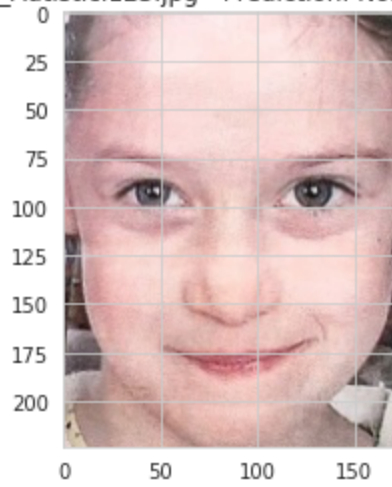


Figure 15. Prediction Example 7

Non_Autistic.24.jpg - Prediction: Non-Autistic

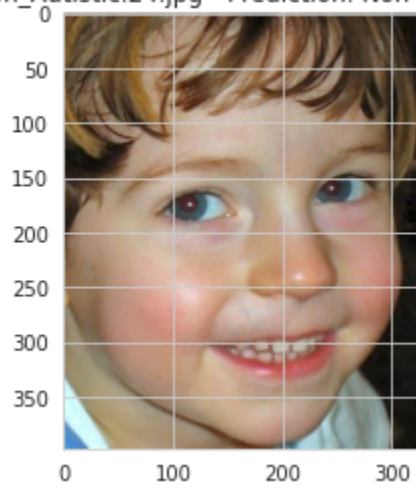
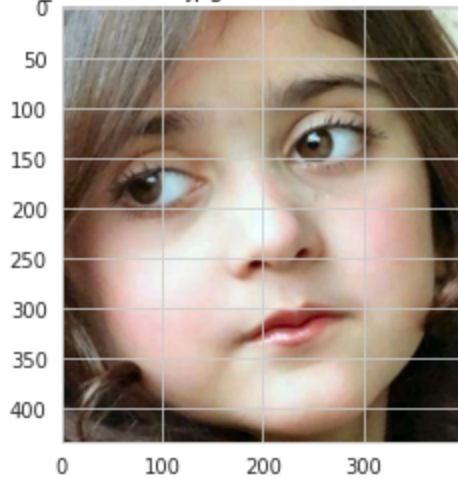


Figure 16. Prediction Example 8

Non_Autistic.47.jpg - Prediction: Non-Autistic



While these figures only provide a small sample of the predictions done to showcase the prediction output, the majority of the predictions given by the model were correct which means that there is a correlation between machine learning and autism diagnosis.

Final Touches

Further improvements to the accuracy of the model are currently ongoing. The trained model was then saved, and a program was written to facilitate the use of the model on a single-image diagnosis basis. The Flask¹⁸ web framework was used to write the program and web server that performs the diagnosis. The program was written in Python with Anaconda Navigator for Windows and Notebook++. The test app was loaded onto a website, www.abstractlogicaljourneys.com/autism, for testing purposes. Single images from the autism data set were submitted to the app to test the program functionality. The code and app pages are shown below.

Submit Page

Index.html - Submit Page

```
<html>

<head>

<link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css">

  <title>X - Autism Diagnosis Test Research</title>

</head>

<body>

  <div class="container">

    <div class="row">

      <div class="col-lg-12">

        <center><h1 style="padding:30px;font-weight: 600 !important;font-size: 3.2em">Autism
Diagnostic Tool</h1></center>

      </div>


      <center><form action = "/predict" method = "post" enctype="multipart/form-data">

        <input type="file" name="file" />

        <input type = "submit" value="Upload">

      </form></center>

    </div>

  </body>

</html>
```

Predict.html - Results Page

```
<!-- Required meta tags -->

<meta charset="utf-8">

<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

<link rel="stylesheet"

href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css">

<title>X - Autism Diagnosis Research</title>

</head>

<body>

<div class="container">

<div class="col-lg-12">

<center><h1 style="padding:30px;font-weight: 600 !important;font-size: 3.2em">Autism

Diagnostic Tool</h1></center>

</div>

<div class="col-lg-12">

<center><form class="form-inline" action = "/predict" method = "post"

enctype="multipart/form-data">

<input type="file" name="file"/><br>

<input type = "submit" class="btn btn-success" value="Predict">

</form></center>

</div>

<div class="col-lg-5" style="padding-top:30px">
```

```

<span class="border border-primary">

</span>

</div>

<div class="col-lg-5" style="padding-top:30px">

    <h4>Subject is <mark
style="background-color:#04aa6d;color:white;border-radius:5px">{{ Result }}</mark></h4>

    <h4>Probabilities : {{ prob }}</h4>

</div>

</div>

</div>

</body>

</html>

```

App.py - model prediction program

```

from flask import Flask, render_template, request

from waitress import serve

from tensorflow.keras.models import load_model

import numpy as np

from tensorflow.keras.preprocessing.image import load_img

from tensorflow.keras.applications.vgg16 import preprocess_input

import os

```

```

from tensorflow.keras.preprocessing import image

app = Flask(__name__)

model = load_model('vgg_model50bin.h5')

target_img = os.path.join(os.getcwd() , 'static/images')

@app.route('/')

def index_view():

    return render_template('index.html')

#Allow files with extension png, jpg and jpeg

ALLOWED_EXT = set(['jpg' , 'jpeg' , 'png'])

def allowed_file(filename):

    return '.' in filename and \

        filename.rsplit('.', 1)[1] in ALLOWED_EXT

# Function to load and prepare the image in right shape

def read_image(filename):

    img = load_img(filename, target_size=(224, 224))

    img_array = image.img_to_array(img)

    img_batch = np.expand_dims(img_array, axis=0)

    return img_batch

@app.route('/predict',methods=['GET','POST'])

def predict():

    if request.method == 'POST':

        file = request.files['file']

```

```

if file and allowed_file(file.filename): #Checking file format

    filename = file.filename

    file_path = os.path.join('static/images', filename)

    file.save(file_path)

    img = read_image(file_path) #prepressing method

    #class_prediction=(model.predict(img) > 0.5).astype("int32")

    classes_x=(model.predict(img) > 0.5).astype("int32")

    if classes_x == 0:

        Result = "Non-Autistic"

    elif classes_x == 1:

        Result = "Autistic"

    else:

        Result = "Alien"

    #'fruit' , 'prob' . 'user_image' these names we have seen in predict.html.

    return render_template('predict.html', Result = Result, prob = classes_x, user_image =
file_path)

    else:

        return "Unable to read the file. Please check file extension"

if __name__ == '__main__':

    app.run(host="0.0.0.0",debug=True,use_reloader=True,port=8080)

    #serve(app, host='0.0.0.0', port=8080, url_scheme='http')

```

These sets of code create the simple interface that is utilized by the user. The two interface screens will be shown in the figures below.

Figure 17. Interface (1)



This screen, although bare, is the one that users are greeted with upon entering the program. To use it, the user just has to upload an image file using the corresponding button. After the user uploads an image file, the 'Upload' button will signal the program to perform a prediction. The next screen, also simple in design, will display the results.

Figure 18. Interface (2)

1/3/24, 2:57 AM

Catherine - Autism Diagnosis Research

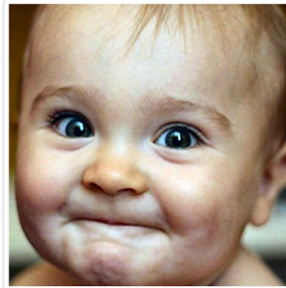
Autism Diagnostic Tool

Choose File No file chosen

Predict

Subject is Autistic

Probabilities : [[1]]



Limitations and Implications

Limitations

This study was affected by limitations that stem from a lack of time and available options. First, the materials used in studies similar to the work of Rahman and Subashini were expensive and unavailable to a student researcher. Machine learning models are resource-heavy, which means that they require highly efficient computers that most student researchers would not have. In this research, the model was coded on a device that would be unable to process the data fast enough. To remedy this, servers available to the public were used to run the model instead. While these servers were highly useful in this project, they often had periods of long run times during peak usage hours of the day. This factor played a large part in the developmental process of the project, as it had to be done in a short time frame that is not comparable to the likes of professional researchers. Within this timeframe, a student only has a limited amount of time during the day to work on such a project because of the hours that school takes up. Not only that, but many high schoolers have large amounts of homework after school to do as well. This project will not seem as nuanced as projects done in devoted research institutions because of the lack of time. Another limitation of this project included the lack of available datasets. Noted by multiple works within the field, there is only one available image set that can be used in research projects such as this. The lack of varied data in a project like this can lead the machine learning model to provide biased or inaccurate answers. This limitation was only made more prominent since testing on real subjects was not available for this work due to safety and privacy concerns. Testing the app would have to be done in the presence of a pediatrician who approved the

project, however, there were no pediatricians able to sign off on a test like this at the time of the model's training.

Implications

Despite the limitations, this study explores an alternative method of promoting machine learning use for autism diagnosis. Researchers like Simeoli have already found results with the use of pre-trained machine learning models and autism diagnosis involving the use of expensive technology. However, this study focused on using the same machine learning technology to create a form of autism diagnosis that is more accessible to the public. To do this, a mobile app was deemed the most accessible form for the model to be created. A mobile app that can lead to more efficient autism diagnosis would allow for more than thousands of children to have access to materials early on. Accessibility on a larger scale is important because many resources for autistic children are time-dependent, meaning that the child needs to be given access to the resource as early as possible. This study acts as a gateway into more research on providing accessible autism diagnosis. With more accessible diagnoses, millions of autistic people would be provided with the programs, therapy, and other resources that they have been unable to access in the past.

Bibliography

1. Maenner MJ. Prevalence and Characteristics of Autism Spectrum Disorder Among Children Aged 8 Years — Autism and Developmental Disabilities Monitoring Network, 11 Sites, United States, 2020. *MMWR Surveillance Summaries*. 2023;72(2). doi:<https://doi.org/10.15585/mmwr.ss7202a1>
2. Verbanas P. One-Fourth of Children with Autism Are Undiagnosed. www.rutgers.edu. Published January 9, 2020. Accessed September 18, 2023.

<https://www.rutgers.edu/news/one-fourth-children-autism-are-undiagnosed#:~:text=Researchers%20analyzed%20the%20education%20and>

3. Mandy W, Midouhas E, Hosozawa M, Cable N, Sacker A, Flouri E. Mental health and social difficulties of late-diagnosed autistic children, across childhood and adolescence. *Journal of Child Psychology and Psychiatry*. 2022;63(11). doi:<https://doi.org/10.1111/jcpp.13587>
4. Halfon N, Stevens GD, Larson K, Olson LM. Duration of a Well-Child Visit: Association With Content, Family-Centeredness, and Satisfaction. *Pediatrics*. 2011;128(4):657-664. doi:<https://doi.org/10.1542/peds.2011-0586>
5. Qiang J, Wu D, Du H, Zhu H, Chen S, Pan H. Review on Facial-Recognition-Based Applications in Disease Diagnosis. *Bioengineering*. 2022;9(7):273. doi:<https://doi.org/10.3390/bioengineering9070273>
6. Simeoli R, Milano N, Rega A, Marocco D. Using Technology to Identify Children With Autism Through Motor Abnormalities. *Frontiers in Psychology*. 2021;12. doi:<https://doi.org/10.3389/fpsyg.2021.635696>
7. Mujeeb Rahman KK, Subashini MM. Identification of Autism in Children Using Static Facial Features and Deep Neural Networks. *Brain Sciences*. 2022;12(1):94. doi:<https://doi.org/10.3390/brainsci12010094>
8. Eslami T, Almuqhim F, Raiker JS, Saeed F. Machine Learning Methods for Diagnosing Autism Spectrum Disorder and Attention- Deficit/Hyperactivity Disorder Using Functional and Structural MRI: A Survey. *Frontiers in Neuroinformatics*. 2021;14. doi:<https://doi.org/10.3389/fninf.2020.575999>
9. Kraft C, Badesch S, Shannon J, et al. *Wait Times and Processes for Autism Diagnostic Evaluations: A First Report Survey of Autism Centers in the U.S.*; 2023. <https://www.cms.gov/files/document/wait-times-and-processes-autism-diagnostic-evaluations-first-report-survey-autism-centers-us.pdf>
10. Kane S, Jayant C, Wobbrock J, Ladner R. *Freedom to Roam: Mobile Device Opportunities and Challenges for People with Visual and Motor Impairments.*; 2009. Accessed October 19, 2023. <https://mobileaccessibility.cs.washington.edu/publications/kane-assets09.pdf>
11. Ventola CL. Mobile Devices and Apps for Health Care Professionals: Uses and Benefits. *P & T: a peer-reviewed Journal for Formulary Management*. 2014;39(5):356-364. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4029126/>

12. Zaina LAM, Fortes RPM, Casadei V, Nozaki LS, Paiva DMB. Preventing accessibility barriers: Guidelines for using user interface design patterns in mobile applications. *Journal of Systems and Software*. 2022;186:111213. doi:<https://doi.org/10.1016/j.jss.2021.111213>
13. Baxter C, Carroll JA, Keogh B, Vandelanotte C. Assessment of Mobile Health Apps Using Built-In Smartphone Sensors for Diagnosis and Treatment: Systematic Survey of Apps Listed in International Curated Health App Libraries. *JMIR mHealth and uHealth*. 2020;8(2):e16741. doi:<https://doi.org/10.2196/16741>
14. Kaggle. Autism_Image_Data. www.kaggle.com. Published 2020. Accessed April 22, 2024. <https://www.kaggle.com/datasets/cihan063/autism-image-data>
15. Abadi M, Agarwal A, Barham P, et al. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems*.; 2015. <https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/45166.pdf>
16. Chollet Fran. Keras. [Keras.io](https://keras.io). Published 2019. <https://keras.io>
17. Glusker A. Library Guides: Research Methods--Quantitative, Qualitative, and More: Data Science Methods (Machine Learning, AI, Big Data). guides.lib.berkeley.edu. Accessed September 6, 2023. <https://guides.lib.berkeley.edu/c.php?g=1262657&p=9256364#:~:text=Machine%20learning%20algorithms%20are%20used>
18. Flask. Welcome to Flask — Flask Documentation (3.0.x). flask.palletsprojects.com. Published 2010. <https://flask.palletsprojects.com/en/3.0.x/>

